

Advanced Cryptography - Theory & Implementation Notes

Student: 231040011006

Table of Contents

#	Topic	Algorithm Type
1	NTRU Encryption & Decryption	Post-Quantum
2	Fully Homomorphic Encryption (FHE)	Cloud Security
3	Elliptic Curve Cryptography (ECC)	Public Key
4	Elliptic Curve Diffie-Hellman (ECDH)	Key Exchange
5	Hasse's Theorem	ECC Theory
6	Closest Vector Problem (CVP)	Lattice Problem
7	Shortest Vector Problem (SVP)	Lattice Problem
8	GGH Cryptosystem	Lattice Crypto

1. NTRU Encryption & Decryption

What is NTRU?

NTRU (N-th degree Truncated polynomial Ring Units) is a post-quantum public-key cryptosystem. Unlike RSA (which can be broken by quantum computers), NTRU is believed to be quantum-safe.

Key Idea: Instead of working with plain numbers, NTRU works with polynomials in a special ring, and security comes from the difficulty of solving lattice problems.

📖 Mathematical Background

Polynomial Ring: We work in the ring:

$$R = \mathbb{Z}[x] / (x^N - 1)$$

This means: all polynomials of degree less than N , where multiplication wraps around (i.e., $x^N = 1$).

Parameters: | Symbol | Meaning | |-----|-----| | N | Degree of polynomial ring | | p | Small modulus (e.g., 3) | | q | Large modulus (e.g., 41, must have $\gcd(p,q)=1$) | | f | Private key polynomial (must be invertible mod p and mod q) | | g | Another private polynomial | | r | Random blinding polynomial (for encryption) |

🔑 How Key Generation Works

1. Choose private polynomials f and g with small coefficients (like -1, 0, 1)
2. Compute $F_q = f^{-1} \pmod{q}$ (inverse of f in ring, mod q)
3. Compute $F_p = f^{-1} \pmod{p}$ (inverse of f in ring, mod p)
4. Compute public key: $h = F_q * g \pmod{q}$

Private key = f (and F_p for decryption)

Public key = h

🔒 How Encryption Works

Formula: $c = p * h * r + m \pmod{q}$

Step by step: 1. Compute $h * r \pmod{q}$ 2. Multiply by p : $p * h * r \pmod{q}$ 3. Add message: $c = p * h * r + m \pmod{q}$

The random r hides the message (like adding random noise).

🔓 How Decryption Works

Step 1: $a = f * c \pmod{q}$

Step 2: Center lift a (bring values to range $[-q/2, q/2]$)

Step 3: $a = a \pmod{p}$

Step 4: $m = F_p * a \pmod{p}$

Step 5: Center lift to get final message

Why does this work?

$$\begin{aligned} f * c &= f * (p * h * r + m) \\ &= f * p * F_q * g * r + f * m && [\text{since } h = F_q * g] \\ &= p * g * r + f * m && [\text{since } f * F_q = 1] \end{aligned}$$

When we reduce mod q then mod p , the $p \cdot g \cdot r$ term vanishes (divisible by p), leaving $f \cdot m$. Then multiplying by Fp gives us m .

Code Implementation

Code file: 1-NTRU_Encryption_Decryption.py

Function	What it does
<code>poly_add(a, b, N, mod)</code>	Adds two polynomials mod q
<code>poly_mul(a, b, N, mod)</code>	Multiplies polynomials mod $(x^N - 1)$ and mod q
<code>center_lift(poly, mod)</code>	Brings values to range $[-\text{mod}/2, \text{mod}/2]$
<code>poly_inv(f, N, mod)</code>	Finds polynomial inverse using Extended GCD
<code>keygen(N, p, q, f, g)</code>	Generates public and private keys
<code>encrypt(m, r, h, N, p, q)</code>	Encrypts a message polynomial
<code>decrypt(c, f, Fp, N, p, q)</code>	Decrypts back to message

Example Inputs (Test Values)

```
N = 7
p = 3
q = 41
f = -1 1 1 0 -1 0 1
g = 1 0 1 1 0 1 0
m = 0 1 0 0 1 -1 0
r = 1 -1 0 1 0 0 0
```

Security

- Security is based on the NTRU lattice problem — a variant of CVP/SVP
 - Considered quantum-resistant (NSA-approved family of algorithms)
 - Key size is much smaller than RSA for same security level
-

2. Fully Homomorphic Encryption (FHE)

What is Homomorphic Encryption?

Homomorphic Encryption allows you to compute on encrypted data without ever decrypting it.

Real World Example: You upload encrypted medical records to a cloud server. The server computes statistics on your data WITHOUT ever learning your private health information.

Types:	Type	What it allows	Partially Homomorphic
Only addition OR only multiplication			Somewhat Homomorphic
Both, but limited number of operations			Both, but limited number of operations
Unlimited additions and multiplications			Fully Homomorphic

How Our Simple FHE Works

This is a toy/simplified version to understand the concept.

Encryption Formula:

$$c = s * (r * q + m)$$

Where: - m = single digit message (0-9) - s = secret key - r = random number (adds noise/randomness) - q = small modulus

Decryption Formula:

Step 1: $temp = (c * s_inv) \% p$

Step 2: $result = temp \% q$

Where s_inv is the modular inverse of s under p .

Key Generation

s = secret key (any integer)

p = large modulus (prime, larger than $s * q$)

q = small modulus (e.g., 10 for digit messages)

Why Does Decryption Work?

$$\begin{aligned} \text{Decrypt}(c) &= \text{Decrypt}(s * (r * q + m)) \\ &= (c * s_inv) \% p \quad \rightarrow \text{removes } s \rightarrow \text{gives } (r * q + m) \\ &= result \% q \quad \rightarrow \text{removes } r * q \rightarrow \text{gives } m \end{aligned}$$

The $\%p$ removes the s , and $\%q$ removes the random noise $r * q$.

Code Implementation

Code file: 2-Fully_Homomorphic_Encryption.py

Function	What it does
<code>mod_inverse(a, mod)</code>	Finds x such that $(a \cdot x) \% \text{mod} == 1$
<code>keygen()</code>	Takes s, p, q from user
<code>encrypt(m, s, q)</code>	Encrypts one digit
<code>decrypt(c, s, p, q)</code>	Decrypts one ciphertext
<code>encrypt_message(message, s, q)</code>	Encrypts all digits
<code>decrypt_message(cipher_list, s, p, q)</code>	Decrypts all ciphertexts

Example Inputs

```
s = 3
p = 71
q = 10
message = 1234
```

3. Elliptic Curve Cryptography (ECC)

What is an Elliptic Curve?

An elliptic curve is defined by the equation:

$$y^2 = x^3 + ax + b \pmod{p}$$

It's NOT an ellipse! It's a special curve with a mathematical group structure — you can “add” points on it.

Why use curves? For the same security level, ECC needs much smaller keys than RSA. A 256-bit ECC key $\approx$ 3072-bit RSA key.

Point Addition Rules

Given two points $P = (x1, y1)$ and $Q = (x2, y2)$ on the curve:

Case 1: $P \neq Q$ (two different points)

```
slope m = (y2 - y1) * (x2 - x1)^(-1) mod p
x3 = m^2 - x1 - x2 mod p
y3 = m*(x1 - x3) - y1 mod p
Result = (x3, y3)
```

Case 2: $P = Q$ (point doubling)

```
slope m = (3*x12 + a) * (2*y1)(-1) mod p
x3 = m2 - 2*x1 mod p
y3 = m*(x1 - x3) - y1 mod p
```

Special Cases: $-P + O = P$ (O is the “point at infinity”, identity element) $-P + (-P) = O$ (negative point has same x , negated y)

🔑 Key Generation (ECC ElGamal)

Private key: d (a random secret number)

Public key: $Q = d * G$ (scalar multiplication of base point)

G is a publicly known generator/base point on the curve.

🔒 Encryption

Input: Message point M , random k

$C1 = k * G$

$C2 = M + k * Q$

Ciphertext = $(C1, C2)$

🔓 Decryption

$dC1 = d * C1$

$M = C2 - dC1 = C2 + (-dC1)$

Why does this work?

$dC1 = d*(kG) = k*(dG) = k*Q$

$C2 - dC1 = (M + k*Q) - k*Q = M$

💻 Code Implementation

Code file: 3-Elliptic_Curve_Cryptography.py

Function	What it does
<code>mod_inv(a, p)</code>	Modular inverse using Python's <code>pow</code>
<code>point_add(P, Q, a, p)</code>	Adds two curve points
<code>scalar_mult(k, P, a, p)</code>	Multiplies point by scalar (double-and-add)

Function	What it does
<code>is_on_curve(P, a, b, p)</code>	Checks if point satisfies curve equation

Example Inputs

`p = 17, a = 2, b = 2`
`G = (5, 1)`
`Private key d = 2`
`Message point M = (6, 3)`
`Random k = 3`

4. Elliptic Curve Diffie-Hellman (ECDH)

What is ECDH?

ECDH is a key exchange protocol that lets two people agree on a shared secret key over an insecure channel — without ever sending the secret itself.

Analogy: Alice and Bob mix paint. They both start with a common color (public). Each adds their secret color (private). When mixed together, both get the same final color (shared secret). An observer only sees the intermediate mixed colors, not the secrets.

How ECDH Works

Step 1: Public Setup Both parties agree on: - Curve parameters: `p`, `a`, `b` - Generator point: `G`

Step 2: Key Generation

Alice: chooses private key `dA` → computes `QA = dA * G` (public)
 Bob: chooses private key `dB` → computes `QB = dB * G` (public)

Step 3: Key Exchange

Alice computes: `S = dA * QB`
 Bob computes: `S = dB * QA`

Step 4: Both get the SAME secret!

$dA * QB = dA * (dB * G) = dA * dB * G$
 $dB * QA = dB * (dA * G) = dA * dB * G$

Why is it secure? An attacker sees QA, QB and G, but cannot compute dA or dB. This is the Elliptic Curve Discrete Logarithm Problem (ECDLP) — computationally infeasible for large curves.

Code Implementation

Code file: 4-ECDH.py

Uses same `point_add` and `scalar_mult` as ECC (file 3), applied for key exchange.

Flow: 1. Input curve parameters and G 2. Alice inputs her private key dA → compute QA 3. Bob inputs his private key dB → compute QB 4. Compute SA = dA * QB and SB = dB * QA 5. Verify SA == SB (shared secret matches)

Example Inputs

```
p = 17, a = 2, b = 2
G = (5, 1)
Alice's dA = 2
Bob's dB = 3
```

5. Hasse's Theorem

What is Hasse's Theorem?

Hasse's Theorem bounds the number of points on an elliptic curve over a finite field.

Statement: > If E is an elliptic curve over a finite field with p elements (Fp), and N is the number of points on E (including the point at infinity), then: > > $|N - (p + 1)| \leq 2\sqrt{p}$

Mathematical Meaning

Define:

$t = (p + 1) - N$ (called Trace of Frobenius)

Hasse's Theorem says:

$$|t| \leq 2\sqrt{p}$$

This means:

$$p + 1 - 2\sqrt{p} \leq N \leq p + 1 + 2\sqrt{p}$$

N is always “close” to $p+1$, within a window of size $4\sqrt{p}$.

Why Does This Matter?

1. Cryptographic security: We need the curve order N to be prime (or have a large prime factor). Hasse’s theorem guarantees $N \approx p$, so large primes exist.
 2. Algorithm efficiency: Knowing N is approximately p helps estimate computation costs.
 3. Curve selection: Curves where N is exactly prime are called prime-order curves and are preferred in cryptography.
-

Code Implementation

Code file: 5-Hasse's_Theorem.py

Function	What it does
<code>count_points(a, b, p)</code>	Brute force: find all points on curve
<code>print_hasse(a, b, p)</code>	Displays theorem verification step by step

Algorithm for counting points:

```
For each x in 0..p-1:
    Compute rhs = (x3 + ax + b) mod p
    For each y in 0..p-1:
        If y2 mod p == rhs:
            Add (x, y) to points list
N = len(points) + 1    (add 1 for point at infinity)
```

Example Inputs

```
p = 23
a = 1
b = 1
```

Expected: N is between $p+1-2\sqrt{p}$ and $p+1+2\sqrt{p}$.

6. Closest Vector Problem (CVP)

What is a Lattice?

A lattice is a set of points in space formed by all integer combinations of basis vectors.

Lattice = $\{ a_1*b_1 + a_2*b_2 + \dots + a_n*b_n \mid a_1, a_2, \dots, a_n \}$

Example:

Basis: $b_1 = [2, 0]$, $b_2 = [0, 3]$

Lattice points: $(0,0)$, $(2,0)$, $(-2,0)$, $(0,3)$, $(2,3)$, $(4,-3)$, ...

The CVP Problem

Given: - A lattice (defined by its basis) - A target vector w (NOT necessarily in the lattice)

Find: - The lattice vector v closest to w

This is NP-hard in general — no efficient algorithm exists for large dimensions!

Babai's Rounding Algorithm

Babai's algorithm finds an approximate closest vector (not always exact):

Step 1: Solve $B*x = w \rightarrow x = B^{-1} * w$ (real-valued coefficients)

Step 2: Round each x_i to nearest integer $\rightarrow$ rounded_x

Step 3: Compute $v = B * \text{rounded_x}$ (this is our approximate closest vector)

Visual intuition:

w = some target point

$B^{-1}*w$ = coordinates of w in the basis coordinate system

Round $\rightarrow$ snap to nearest lattice coordinate

Reconstruct $\rightarrow$ gives nearest lattice point

Code Implementation

Code file: 6-Closest_Vector_Problem.py

Function/Step	What it does
Build matrix B (columns = basis vectors)	Organizes basis
<code>np.linalg.solve(B, w)</code>	Solves $B*x = w$
<code>np.round(coeffs)</code>	Rounds to nearest integers

Function/Step	What it does
B @ rounded	Reconstructs lattice vector
norm(w - v)	Measures distance

Example Inputs

```
n = 2
v1: 137 312
v2: 215 -187
w: 53172 81743
```

Cryptographic Relevance

CVP is the foundation of GGH cryptosystem security: - Encryption adds small noise to a lattice point - Decryption = solve CVP using a “good” private basis - Attacker only has the “bad” public basis → CVP is hard

7. Shortest Vector Problem (SVP)

What is SVP?

Given: - A lattice (defined by its basis)

Find: - The non-zero lattice vector with the smallest Euclidean length

SVP: Find $v \in \text{Lattice}$, $v \neq 0$, minimizing $\|v\|$

Why is SVP Hard?

In 2D, you can visualize and solve it easily. But as dimension grows: - Number of lattice vectors grows exponentially - Best known algorithms run in exponential time - This hardness is the basis for post-quantum cryptography!

Our Brute Force Approach

For small dimensions, we try all integer combinations:

```
For all (c1, c2, ..., cn) where ci in [-limit, limit]:
    Compute v = c1*b1 + c2*b2 + ... + cn*bn
```

```

    Compute  $\|v\|$ 
    Track minimum  $\|v\|$ 

```

Complexity: - Number of combinations = $(2 \cdot \text{limit} + 1)^n$ - For limit=3, n=4: $7^4 = 2401$ combinations - For limit=3, n=50: $7^{50} \approx 10^{42}$ combinations (impossible!)

Code Implementation

Code file: 7-SHORTEST_VECTOR_PROBLEM.py

Function	What it does
<code>dot_product(v, w)</code>	Computes $v \cdot w$
<code>vector_add(v, w)</code>	Computes $v + w$
<code>scalar_mult(a, v)</code>	Computes $a * v$
<code>vector_norm(v)</code>	Computes $\ v\ $
<code>generate_lattice_vectors(basis, limit)</code>	Generates all lattice vectors in range limit
<code>solve_svp(basis, limit)</code>	Finds shortest vector

Example Inputs

```

n = 2
v1: 31 59
v2: 37 70
limit: 3

```

Cryptographic Relevance

- NTRU security reduces to SVP
- Learning With Errors (LWE) also related to SVP
- All major post-quantum cryptography standards (NIST 2024) are based on lattice problems (SVP/CVP hardness)

8. GGH Cryptosystem

What is GGH?

GGH (Goldreich-Goldwasser-Halevi, 1997) is a lattice-based public-key cryptosystem.

It's historically important because it was the first practical lattice cryptosystem, though it was later broken for real-world parameters.

🔑 Core Idea: Two Bases, Same Lattice

The trick is having TWO bases for the SAME lattice:

Basis	Type	Who has it	Properties
V	Good (private key)	Owner	Nearly orthogonal, easy to solve CVP
W	Bad (public key)	Everyone	Messy, hard to solve CVP

A unimodular matrix U ($\det = \pm 1$) transforms the good basis into the bad one:

$$W = U * V$$

Because $\det(U) = \pm 1$, U preserves the lattice (integer invertible).

🔑 Key Generation

1. Choose good basis V (private key)
2. Choose unimodular matrix U ($\det = \pm 1$)
3. Compute $W = U * V$ (public key / bad basis)

Good Basis Properties: - Vectors should be nearly orthogonal (large angles between them) - Short vectors - Makes Babai's algorithm work well

🔒 Encryption

$$\text{Formula: } c = m * W + r$$

Where: - m = message vector (integers) - W = public bad basis - r = small random noise vector (e.g., entries $\in \{-1, 0, 1\}$)

The ciphertext c is a lattice point ($m*W$) perturbed by noise r .

Decryption

Step 1: Use Babai's algorithm with PRIVATE good basis V to find closest lattice point:

```
v = Babai(V, c)
```

This removes the noise r from c , giving the exact lattice point $m \cdot W$.

Step 2: Recover message by inverting W :

```
m = v * W-1
```

Why Can Only the Owner Decrypt?

- Owner has good basis $V \rightarrow$ Babai's algorithm works $\rightarrow$ CVP solved $\rightarrow$ noise removed $\rightarrow$ m recovered
 - Attacker only has bad basis $W \rightarrow$ Babai's algorithm fails $\rightarrow$ noise can't be removed $\rightarrow$ m not recovered
-

Code Implementation

Code file: 9-GGH_CRYPTOSYSTEM.py

Function	What it does
<code>babai(B, w)</code>	Babai's rounding algorithm for CVP
Main	Full GGH: key gen, encrypt, decrypt, verify

Steps in code:

1. Input private basis V
 2. Input unimodular matrix U
 3. Compute $W = U @ V$ (numpy matrix multiply)
 4. Input message m and noise r
 5. Encrypt: $c = m @ W + r$
 6. Decrypt: $v = \text{babai}(V, c)$ then $m = v @ W^{-1}$
 7. Verify: $m == m_{\text{recovered}}$
-

Example Inputs

```
n = 2
```

```
Private basis:
```

v1: 4 13
v2: -57 -45

Unimodular matrix U:
2 3
3 5

Message m:
2 -5

Noise r:
1 -1

🌐 Relationships Between Topics

-----+	-----+
LATTICE THEORY	ELLIPTIC CURVES
-----+	-----+
SVP (File 7) <---+---hard	ECC (File 3)
CVP (File 6) <---+---hard	ECDH (File 4)
GGH (File 9)	Hasse (File 5)
uses CVP for	Both based on
encryption	discrete log
NTRU (File 1)	
uses lattice	-----+
ring problems	ADVANCED CRYPTO
-----+	-----+
	FHE (File 2)
	compute on
	encrypted data
	-----+

⚡ Quick Reference: All Example Inputs

File	Key Test Inputs
1-NTRU	$N=7, p=3, q=41, f=-1 \ 1 \ 1 \ 0 \ -1 \ 0 \ 1, g=1 \ 0 \ 1 \ 1 \ 0 \ 1 \ 0, m=0 \ 1 \ 0 \ 0 \ 1 \ -1 \ 0, r=1 \ -1 \ 0 \ 1 \ 0 \ 0 \ 0$
2-FHE	$s=3, p=71, q=10, \text{message}=1234$
3-ECC	$p=17, a=2, b=2, G=(5,1), d=2, M=(6,3), k=3$
4-ECDH	$p=17, a=2, b=2, G=(5,1), dA=2, dB=3$
5-Hasse	$p=23, a=1, b=1$
6-CVP	$n=2, v1=137 \ 312, v2=215 \ -187, w=53172 \ 81743$
7-SVP	$n=2, v1=31 \ 59, v2=37 \ 70, \text{limit}=3$
9-GGH	$n=2, V=(4 \ 13, \ -57 \ -45), U=(2 \ 3, \ 3 \ 5), m=2 \ -5, r=1 \ -1$

Key Terms Glossary

Term	Meaning
Lattice	Grid of points formed by integer combinations of basis vectors
Basis	Set of independent vectors that define the lattice
Unimodular matrix	Integer matrix with determinant ± 1
SVP	Shortest Vector Problem — find shortest non-zero lattice vector
CVP	Closest Vector Problem — find lattice vector nearest to target point
Babai's Algorithm	Approximation algorithm for CVP using rounding trick
Scalar multiplication	Adding a point to itself k times on elliptic curve
Point at infinity (O)	Identity element in elliptic curve group
Generator point G	A basepoint used to derive public keys on elliptic curve
Trace of Frobenius	$t = (p+1) - N$, used in Hasse's theorem
Polynomial ring	Ring of polynomials with coefficients mod q
Homomorphic	"Same structure" — operations on ciphertext = operations on plaintext
Post-quantum	Cryptography believed secure against quantum computers
Modular inverse	$a^{-1} \bmod n$ such that $a * a^{-1} \equiv 1 \pmod{n}$
ECDLP	Elliptic Curve Discrete Log Problem — given $Q=kG$, find k (hard!)

Generated for Cryptography Assignment | Roll No: 231040011006